P1035R2
Input range adaptors

Published Proposal, 2018-10-08

This version:
https://wg21.link/p1035

Authors:
Christopher Di Bella
Casey Carter
Corentin Jabot

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1035 proposes to introduce seven additional range adaptors in the C++20 timeframe.

Table of Contents

§ 1. Changes

§ 1.1. From R1 to R2

- Expanded acknowledgements and co-authors.
- Removed `zip_with_view`.
- Added `zip_view`.
- Added `keys` and `values`.
- Added content for associated types for ranges.

§ 1.2. From R0 to R1

- Revised `istream_range`.
- Renamed to `basic_istream_view`.
- Introduced some relevant concepts.
- Introduced `drop_view`, `take_while_view`, `drop_while_view`.
- Teased `zip_with_view`.
- Teased associated types for ranges.

§ 2. Acknowledgements

We would like to acknowledge the following people for their assistance with this proposal:

- Eric Niebler, for providing [range-v3] as a reference implementation.
- Tim Song, Steve Downey, and Barry Revzin for their reviews of P1035.

§ 3. Motivation

[P0789] introduces the notion of a range adaptor and twelve pioneering range adaptors that improve declarative, range-based programming. For example, it is possible to perform an inline, in-place, lazy reverse like so:

```
namespace ranges = std::ranges;
namespace view = std::ranges::view;

// Old
auto i = ranges::find(ranges::rbegin(employees), ranges::rend(employees), "Lovelace", &empl

// New
auto j = ranges::find(employees | view::reverse, "Lovelace", &employee::surname);
[[assert: i == j]];
```

P1035 recognises that P0789 introduces only a few of the widely experimented-with range adaptors in [range-v3], and would like to add a few more to complete the C++20 phase of range adaptors.

§ 4. Proposals

Unless otherwise requested, each sub-subsection below should be polled individually from other sub-subsections. Two major questions are to be asked per range adaptor. It is up to LEWG to decide the exact phrasing, but the author's intentions are listed below.

1. Do we want this range adaptor in C++20?
    1. As-is?
    2. With modifications, as suggested by LEWG?
2. If we do not want this range adaptor in C++20, do we want it in C++23?
    1. As-is?
    2. With modificaitons, as suggested by LEWG?

§ 4.1. `zip_view`

Note: This section was formerly titled *`zip_with_view`* in P1035R1. Due to complications in the design process, `zip_with_view` will not be proposed, but `zip_view` shall be. A subsubsection below articulates how to emulate `zip_with_view` below.

§ 4.1.1. Motivation

A zip, also known as a [convolution] operation, performs a transformation on multiple input ranges. The typical zip operation transforms several input ranges into a single input range containing a tuple with the *i*th element from each range, and terminates when the smallest finite range is delimited.

Iterating over multiple ranges simultaneously is a useful feature. Both [EWG43] and [P0026] ask for this functionality at a language level. While these papers request a useful feature that benefits raw loops, they don't address algorithmic composition, which is important for ensuring both correctness and simplicity. A `zip_view` will allow programmers to iterate over multiple ranges at the same time when requiring either a raw loop or an algorithm.

Current (C++17)

```
auto x = vector<float>{/* ... */};
auto y = vector<float>{/* ... */};
// ...
auto xit = begin(x);
auto yit = begin(y);
for (; xit != end(x) and yit != end(y); ++xit, (void)++yit) {
   // ...
}
```

Proposed (C++20)

```
auto x = vector<float>{/* ... */};
auto y = vector<float>{/* ... */};
// ...
for (auto xy = view::zip(x, y); auto [x
   // ...
}
```

Another motivating example for using `zip` involves replacing raw loops with algorithms and range adaptors, but still being able to index the operation in question.

Current (C++17)

```
auto v = vector(istream_iterator<int>(cin),
                istream_iterator<int>());
auto weighted_sum = 0;
for (auto i = 0; i < ranges::distance(v); ++i) {
   weighted_sum += i * v[i];
}
```

Proposed (C++20)

```
auto in = view::zip(view::iota(0), istream_view<int>
auto const weighted_sum = accumulate(begin(in), end(
   [](auto const a, auto const b){
      auto const [index, in] = b;
      return a + (index * in);
   });
```

Finally, people who work in the parallelism and heterogeneous industries are not able to take full advantage of the Parallel STL at present due to the limited number of input ranges each algorithm can support. `zip_view` will permit this (see [P0836] §2.1 for how this is beneficial).

The benefits of this proposed approach include:

- More *declared* operations, leading to more declarative -- rather than imperative -- styled programming.
- Eliminates state.
- A resulting expression can be declared `const` without needing to rely on IILE.
- Temporary storage is eliminated, which helps to improve correctness, clarity, and performance. P0836 §2.1 expands on the performance benefits for heterogeneous programming.

§ *4.1.1.1. `zip_with` in C++20*

`zip_with` is a generalisation of `zip`, such that we can apply an *n*-ary function in place of the `transform`s above. The following is an example of how `zip_with` can refine `zip` when tuples are not necessary.

```
vector<float> const x = // ...
vector<float> const y = // ...
float const a = // ...
// ...
auto ax = x | view::transform([a](auto const x) noexcept { return a * x; });
auto saxpy = view::zip_with(plus<>{}, ax, y);
auto const result = vector(begin(saxpy), end(saxpy));
```

As `zip_with_view` is no longer proposed in C++20, users wanting `zip_with` functionality will be required to use something akin to:

```
template<class F>
constexpr auto compose_apply(F&& f) {
   return [f = std::forward<F>(f)](auto&& t) {
      return std::apply(std::forward<F>(f), std::forward<decltype(t)>(t));
   };
};

// ...
auto saxpy = zip(ax, y) | transform(compose_apply(plus<>{}));
```

This isn't an ideal approach, but some of the finer details of `zip_with_view` that are independent of `zip_view` are still being worked out, and this should not preclude the usage of zipping ranges for maximum composability.

§ 4.1.2. Problems with `pair` and `tuple`

`zip_view` requires a value type to store the values of the iterators to the ranges that are iterated over, and a reference type to access those values via an iterator to the range. Additionally, the concept "`Writable` wants `const` proxy references to be assignable, so we need a tuple-like type with `const`-qualified assignment operators"[cmcstl2].

Both range-v3 and the cmcstl2 implementation above use an exposition-only type derived from `tuple` (both) or `pair` (range-v3 only) that permits a `CommonReference` between `tuple<Ts...>&` (an lvalue reference to the value type) and `tuple<Ts&...>` (the perceived reference type). This is deemed to be a hack by both Eric Niebler and Casey Carter; a careful reader might notice that the above implementation specifies this as `__tuple_hack` (which has conveniently been left out of P1035).

Adding a third (and possibly fourth) tuple type that is exposition-only is *not* ideal: this requires extensions to both `pair` and `tuple` so that they are compatible with `__tuple_hack` and `__pair_hack`, specialisations for all tuple utilities will be required, and overloads to `get`, `apply`, etc. are also necessary.

Alternatively, we can provide an implicit conversion from `tuple<Ts...>&` to `tuple<Ts&...>`, and ensure that a `common_reference` exists (à la `basic_common_reference`), and similarly for the other reference types.

```
template<class... Ts>
struct tuple {
   // ...

   constexpr tuple const& operator=(tuple<_Elements...> const&)
   requires (__stl2::Assignable<_Elements const&, _Elements const&> and ...);

   constexpr tuple const& operator=(tuple<_Elements...>&&)
   requires (__stl2::Assignable<_Elements const&, _Elements> and ...);

   template<class... Us>
   requires sizeof...(Ts) == sizeof...(Us) and (Assignable<Ts const&, Us const&> and ...)
   constexpr tuple const& operator=(tuple<Us...> const& other) const;

   template<class... Us>
   requires sizeof...(Ts) == sizeof...(Us) and (Assignable<Ts const&, Us const&> and ...)
   constexpr tuple const& operator=(tuple<Us...>&& other) const;

   constexpr operator tuple<remove_reference_t<_Elements>&...>() noexcept;
   constexpr operator tuple<remove_reference_t<_Elements> const&...>() const noexcept;
   constexpr operator tuple<remove_reference_t<_Elements>&&...>() noexcept;
   constexpr operator tuple<remove_reference_t<_Elements> const&&...>() const noexcept;
};

template<class... Ts>
requires (requires { typename common_type_t<Ts, Ts&>; } && ...)
struct common_type<tuple<Ts&...>, tuple<Ts...>> {
   using type = tuple<common_type_t<Ts&, Ts>...>;
};
template<class... Ts>
requires (requires { typename common_type_t<Ts, Ts&>; } && ...)
struct common_type<tuple<Ts...>, tuple<Ts&...>>   {
   using type = tuple<common_type_t<Ts&, Ts>...>;
};

template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts>, UQual<Ts&>>; } && ...)
struct basic_common_reference<tuple<Ts&...>, tuple<Ts...>, TQual, UQual> {
   using type = tuple<common_reference_t<TQual<Ts&>, UQual<Ts>>...>;
};
template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts>, UQual<Ts&>>; } && ...)
struct basic_common_reference<tuple<Ts...>, tuple<Ts&...>, TQual, UQual> {
   using type = tuple<common_reference_t<TQual<Ts>, UQual<Ts&>>...>;
};

// lvalue to rvalue reference
template<class... Ts>
requires (requires { typename common_type_t<Ts, Ts&&>; } && ...)
struct common_type<tuple<Ts&&...>, tuple<Ts...>> {
   using type = tuple<common_type_t<Ts&&, Ts>...>;
};
template<class... Ts>
requires (requires { typename common_type_t<Ts, Ts&&>; } && ...)
struct common_type<tuple<Ts...>, tuple<Ts&&...>>   {
   using type = tuple<common_type_t<Ts&&, Ts>...>;
};

template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts>, UQual<Ts&&>>; } && ...)
struct basic_common_reference<tuple<Ts&&...>, tuple<Ts...>, TQual, UQual> {
```

```
   using type = tuple<common_reference_t<TQual<Ts&&>, UQual<Ts>>...>;
};
template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts>, UQual<Ts&&>>; } && ...)
struct basic_common_reference<tuple<Ts...>, tuple<Ts&&...>, TQual, UQual> {
   using type = tuple<common_reference_t<TQual<Ts>, UQual<Ts&&>>...>;
};

// lvalue reference to rvalue reference
template<class... Ts>
requires (requires { typename common_type_t<Ts&, remove_reference_t<Ts>&&>; } && ...)
struct common_type<tuple<remove_reference_t<Ts>&&...>, tuple<Ts&...>> {
   using type = tuple<common_type_t<remove_reference_t<Ts>&&, Ts&>...>;
};
template<class... Ts>
requires (requires { typename common_type_t<Ts&, remove_reference_t<Ts>&&>; } && ...)
struct common_type<tuple<Ts&...>, tuple<remove_reference_t<Ts>&&...>>   {
   using type = tuple<common_type_t<remove_reference_t<Ts>&&, Ts&>...>;
};

template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts&>, UQual<remove_reference_t<Ts>&&
struct basic_common_reference<tuple<remove_reference_t<Ts>&&...>, tuple<Ts&...>, TQual, UQu
   using type = tuple<common_reference_t<TQual<remove_reference_t<Ts>&&>, UQual<Ts&>>...>;
};
template<class... Ts,
   template <class> class TQual, template <class> class UQual>
requires (requires { typename common_reference_t<TQual<Ts&>, UQual<remove_reference_t<Ts>&&
struct basic_common_reference<tuple<Ts&...>, tuple<remove_reference_t<Ts>&&...>, TQual, UQu
   using type = tuple<common_reference_t<TQual<Ts&>, UQual<remove_reference_t<Ts>&&>>...>;
};
```

§ *4.1.2.1. Pros and cons for fixing `tuple` (and possibly `pair`)*

Pros	Cons

Exactly one `tuple` and one `pair` exist.	Consider this code:
	`constexpr tuple<int&, double&> bad_code(int i)`
	`{`
	`  auto t = tuple<int, double>{i, 0.0};`
	`  return t;`
	`}`
	
	`int main()`
	`{`
	`   auto const t = bad_code(4); // BOOM! Undefined behaviour.`
	`   auto const u = bad_code(6);`
	`   auto const tu = std::get<0>(t) + std::get<0>(u);`
	`   CHECK(tu == 10);`
	`}`
	
	This program is undefined; GCC's `-Wuninitialized` flag identifies this, and when coupled with `-Werror`, can force a build to fail (this is good if, and only if `-Werror` is enabled for this warning). A naive programmer mightn't be aware that they're calling a conversion operator.
	
	While this may not be motivation for LEWG to avoid making the changes to `tuple` directly, it is certainly important to be aware of this issue.

§ *4.1.2.2. Pros and cons for introducing an exposition-only `__tuple_hack` (and possibly `__pair_hack`)*

Pros	Cons
Dangling issue resolved (users can't directly use exposition-only types).	More library maintainer maintenance.
	More overloads for `std::get`, `tuple_cat`, etc.

§ *4.1.2.3. Ensure `tuple` and `pair` assignment operators are `constexpr`*

To ensure that the `constexpr`-ness of `zip_view` is not accidentally inhibited, this paper also requests that there be some discussion on making the assignment operators of `pair` and `tuple` `constexpr`.

§ **4.1.3. Example implementation**

The following implementation has been taken and modified from cmcstl2:

```
template<View... Rs>
struct zip_view : view_interface<zip_view<Rs...>> {
private: // begin exposition-only
   template <bool Const, class Indices> struct __zipperator;
   template <bool Const, class Indices> struct __sentinel;

   static constexpr bool all_simple = (simple-view<Rs> && ...);
   static constexpr bool all_forward = (ForwardRange<Rs> && ...);
   static constexpr bool all_forward_const = (ForwardRange<const Rs> && ...);
   static constexpr bool all_sized = (SizedRange<Rs> && ...);
   static constexpr bool all_sized_const = (SizedRange<const Rs> && ...);

   tuple<Rs...> ranges_ {};
public:// end exposition-only
   zip_view() = default;

   constexpr zip_view(Rs... rs)
   noexcept((is_nothrow_move_constructible_v<Rs> && ...))
   requires sizeof...(Rs) != 0

   constexpr auto begin();
   constexpr auto begin() const requires all_forward_const;

   constexpr auto end() requires all_forward;
   constexpr auto end() const;

   constexpr auto size() requires all_sized;
   constexpr auto size() const requires all_sized_const;
};

template <class... Rs>
zip_view(Rs&&...) -> zip_view<all_view<Rs>...>;
```

§ *4.1.3.1. `zip_view` constructors*

```
constexpr zip_view(Rs... rs) noexcept(is_nothrow_move_constructible_v<Rs> && ...)
requires (sizeof...(Rs) != 0);
```

1. *Effects*: Initialises `ranges_` with `rs...`.

4.1.3.2. `zip_view` *begin*

```
constexpr auto begin();
constexpr auto begin() const requires all_forward_const;
```

2. *Effects*: Returns an iterator containing a tuple of iterators to the beginning of each range, such that the first iterator corresponds to the beginning of the first range, the second tuple corresponds to the beginning of the second range, and so on.

4.1.3.3. `zip_view` *end*

```
constexpr auto end() requires all_forward;
constexpr auto end() const;
```

3. *Effects*: Returns a sentinel containing a tuple of sentinels to the end of each range, such that the first sentinel corresponds to the end of the first range, the second sentinel corresponds to the end of the second range, and so on.

```
constexpr auto size() requires all_sized;
constexpr auto size() requires all_sized_const;
```

4. *Effects*: Equivalent to returning the size of the smallest range in `ranges_`.

§ *4.1.3.4. `zip_view<Rs...>` iterator (exposition-only)*

```
template<View... Rs>
template<bool Const, size_t... Is>
struct zip_view<Rs...>::__zipperator<Const, index_sequence<Is...>> {
private:
   template<class R>
   using maybe_const = conditional_t<Const, R const, R>;

   static constexpr bool all_random_access =
      (RandomAccessRange<maybe_const<Rs>> && ...);
   static constexpr bool all_bidi =
      (BidirectionalRange<maybe_const<Rs>> && ...);
   static constexpr bool all_forward =
      (ForwardRange<maybe_const<Rs>> && ...);
   static constexpr bool all_input =
      (InputRange<maybe_const<Rs>> && ...);

   static_assert(!Const || all_forward);
   static_assert(Same<index_sequence_for<Rs...>,
      index_sequence<Is...>>);

   zip_view<Rs...>* parent_ = nullptr;
   tuple<iterator_t<maybe_const<Rs>>...> iterators_{};
public:
   using value_type = tuple<iter_value_t<iterator_t<maybe_const<Rs>>>...>;
   using iterator_category = see-below;
   using difference_type = see-below;

   __zipperator() = default;
   explicit constexpr __zipperator(Parent& parent);
   constexpr __zipperator(Parent& parent) requires all_forward;
   constexpr __zipperator(Parent& parent, difference_type n) requires all_random_access;

   constexpr auto operator*();
   constexpr auto operator*() const
   requires (dereferenceable<const iterator_t<maybe_const<Rs>>> && ...);

   constexpr __zipperator& operator++();
   constexpr auto operator++(int);

   constexpr __zipperator& operator--() requires all_bidi;
   constexpr __zipperator operator--(int) requires all_bidi;

   friend constexpr bool operator==(const __zipperator& x, const __zipperator& y)
   requires all_forward;
   friend constexpr bool operator!=(const __zipperator& x, const __zipperator& y)
   requires all_forward;

   friend constexpr bool operator==(const __zipperator& x, default_sentinel) requires !all_
   friend constexpr bool operator==(default_sentinel y, const __zipperator& x) requires !al
   friend constexpr bool operator!=(const __zipperator& x, default_sentinel y) requires !al
   friend constexpr bool operator!=(default_sentinel y, const __zipperator& x) requires !al

   friend constexpr bool operator==(const __zipperator& x, const Sent& y) requires all_forw
   friend constexpr bool operator==(const Sent& y, const __zipperator& x) requires all_forw
   friend constexpr bool operator!=(const __zipperator& x, const Sent& y) requires all_forw
   friend constexpr bool operator!=(const Sent& y, const __zipperator& x) requires all_forw

   friend constexpr bool operator<(const __zipperator& x, const __zipperator& y) requires a
   friend constexpr bool operator>(const __zipperator& x, const __zipperator& y) requires a
   friend constexpr bool operator<=(const __zipperator& x, const __zipperator& y) requires
   friend constexpr bool operator>=(const __zipperator& x, const __zipperator& y) requires
```

```
    constexpr __zipperator& operator+=(difference_type n) requires all_random_access;

    friend constexpr __zipperator operator+(const __zipperator& x, difference_type n) requir
    friend constexpr __zipperator operator+(difference_type n, const __zipperator& x) requir

    constexpr __zipperator& operator-=(difference_type n) requires all_random_access;
    friend constexpr __zipperator operator-(const __zipperator& x, difference_type n) requir
    friend constexpr difference_type operator-(const __zipperator& x, const __zipperator& y)
    requires (SizedSentinel<iterator_t<maybe_const<Rs>>, iterator_t<maybe_const<Rs>>> && ...

    constexpr auto operator[](difference_type n) const requires all_random_access;

    friend constexpr auto iter_move(const __zipperator& x)
    requires ((Readable<iterator_t<maybe_const<Rs>>> && ...));

    friend constexpr void iter_swap(const __zipperator& x, const __zipperator& y);
    requires (IndirectlySwappable<iterator_t<maybe_const<Rs>>> && ...)
};
```

§ *4.1.3.5. `zip_view::__zipperator` members*

```
using iterator_category = see-below;
```

5. *Effects*: If `(RandomAccessRange<Rs> && ...)` is satisfied, then `iterator_category` denotes `random_access_iterator_tag`. Otherwise, if `(BidirectionalRange<Rs> && ...)`, then `iterator_category` denotes `bidirectional_iterator_tag`. Otherwise, if `(ForwardRange<Rs> && ...)`, then `iterator_category` denotes `forward_iterator_category`. Otherwise, if `(InputRange<Rs> && ...)` is satisfied, then `iterator_category` denotes `input_iterator_tag`. Otherwise, `iterator_category` denotes `output_iterator_category`.

```
using difference_type = see-below;
```

6. *Effects*: If `sizeof...(Rs) == 0`, then `difference_type` denotes `int`. Otherwise, `difference_type` denotes `common_type_t<iterator_t<conditional_t<Const, const Rs, Rs>>...>`.

§ *4.1.3.6. `zip_view::__zipperator` constructors*

```
explicit constexpr __zipperator(Parent& parent);
```

7. *Effects*: Initialises `parent_` with `std::addressof(parent)`.

```
constexpr __zipperator(Parent& parent) requires all_forward;
```

8. *Effects*: Initialises `parent_` with `std::addressof(parent)` and `iterators_` with the iterator to the first element in each range (respectively).

```
constexpr __zipperator(Parent& parent, difference_type n) requires all_random_access;
```

9. *Effects*: Initiaises `parent_` with `std::addressof(parent)` and `iterators_` with the *n*th iterator to each range (respectively). For any range `r` in `Rs...`, a program is undefined if `n > size(r)`.

§ 4.1.3.7. `zip_view::__zipperator` element access

```
constexpr auto operator*();
constexpr auto operator*() const;
```

10. *Effects*: Returns a tuple of references to the elements of each range denoted by `*i` for each iterator in the `__zipperator`.

```
constexpr auto operator[](difference_type n) const requires all_random_access;
```

11. *Effects*: Equivalent to `return *(*this + n);`.

§ 4.1.3.8. `zip_view::__zipperator` advance

```
constexpr __zipperator& operator++();
```

9. *Effects*: Equivalent to incrementing each iterator in `iterators_`.
10. *Returns*: `*this`.

```
constexpr auto operator++(int);
```

13. *Effects*: Equivalent to:

```
if constexpr (all_forward) {
   auto temp = *this;
   ++*this;
   return temp;
}
else {
   ++*this;
}
```

```
constexpr __zipperator& operator--() requires all_bidi;
```

14. *Effects*: Equivalent to decrementing each iterator in `iterators_`.

```
constexpr __zipperator operator--(int) requires all_bidi;
```

15. *Effects*: Equivalent to:

```
auto temp = *this;
--*this;
return temp;
```

```
constexpr __zipperator& operator+=(difference_type n) requires all_random_access;
```

16. *Effects*: Equivalent to calling `ranges::advance(E, n)` on each iterator in `iterators_`, where `E` is a placeholder for the iterator expression.

```
constexpr __zipperator& operator-=(difference_type n) requires all_random_access;
```

17. *Effects*: Equivalent to `return *this += -n;`.

§ *4.1.3.9. `zip_view::__zipperator` comparisons*

```
friend constexpr bool operator==(const __zipperator& x, const __zipperator& y) requires all
```

18. *Effects*: Equivalent to `return x.iterators_ == y.iterators_;`.

```
friend constexpr bool operator==(const __zipperator& x, default_sentinel) requires !all_for
friend constexpr bool operator==(default_sentinel y, const __zipperator& x) requires !all_f
```

19. *Effets*: Returns `true` if at least one iterator in `iterators_` is equivalent to the sentinel of its corresponding range, `false` otherwise.

```
friend constexpr bool operator==(const __zipperator& x, const Sent& y) requires all_forward
friend constexpr bool operator==(const Sent& y, const __zipperator& x) requires all_forward
```

20. *Effects*: Returns `true` if at least one iterator in `iterators_` is equivalent to the sentinel.

```
friend constexpr bool operator!=(const __zipperator& x, const __zipperator& y) requires all
friend constexpr bool operator!=(const __zipperator& x, default_sentinel y) requires !all_f
friend constexpr bool operator!=(default_sentinel y, const __zipperator& x) requires !all_f
friend constexpr bool operator!=(const __zipperator& x, const Sent& y) requires all_forward
friend constexpr bool operator!=(const Sent& y, const __zipperator& x) requires all_forward
```

21. *Effects*: Equivalent to `return !(x == y);`.

```
friend constexpr bool operator<(const __zipperator& x, const __zipperator& y) requires all_
```

22. *Effects*: Equivalent to `return x.iterators_ < y.iterators_;`.

```
friend constexpr bool operator>(const __zipperator& x, const __zipperator& y) requires all_
```

23. *Effects*: Equivalent to `return y < x;`

```
friend constexpr bool operator<=(const __zipperator& x, const __zipperator& y) requires all
```

24. *Effects*: Equivalent to `return !(y < x);`

```
friend constexpr bool operator>=(const __zipperator& x, const __zipperator& y) requires all
```

25. *Effects*: Equivalent to `return !(x < y);`

§ *4.1.3.10. `zip_view<Rs...>::__zipperator` arithmetic*

```
friend constexpr __zipperator operator+(const __zipperator& x, difference_type n) requires
friend constexpr __zipperator operator+(difference_type n, const __zipperator& x) requires
```

26. *Effects*: Equivalent to:

```
auto temp = x;
temp += n;
return temp;
```

```
friend constexpr __zipperator operator-(const __zipperator& x, difference_type n) requires
```

27. *Effects*: Equivalent to `return x + -n;`.

```
friend constexpr difference_type operator-(const __zipperator& x, const __zipperator& y)
requires (SizedSentinel<iterator_t<maybe_const<Rs>>, iterator_t<maybe_const<Rs>>> && ...)
```

28. *Effects*: If `sizeof...(Rs) == 0`, then returns 0. Otherwise returns the greatest distance between iterators in `x.iterators_` and `y.iterators_`.

§ *4.1.3.11. `zip_view<Rs...>::__zipperator` iter_move*

```
friend constexpr auto iter_move(const __zipperator& x) requires ((Readable<iterator_t<maybe
```

29. *Effects*: Returns a tuple containing expressions equivalent to `iter_move` applied to each of `x.iterators_`.

§ *4.1.3.12. `zip_view<Rs...>::__zipperator` iter_swap*

```
friend constexpr void iter_swap(const __zipperator& x, const __zipperator& y)
requires (IndirectlySwappable<iterator_t<maybe_const<Rs>>> && ...);
```

30. *Effects*: Equivalent to `iter_swap` applied to each iterator in `x.iterators_` and `y.iterators_`.

§ *4.1.3.13. `zip_view<Rs...>` sentinel (exposition-only)*

```
template<View... Rs>
template<bool Const, size_t... Is>
struct zip_view<Rs...>::__sentinel<Const, index_sequence<Is...>> {
private:
   using Iter = typename zip_view<Rs...>::template
      __zipperator<Const, index_sequence<Is...>>;
   friend Iter;

   using Parent = conditional_t<Const, const zip_view<Rs...>, zip_view<Rs...>>;
   static constexpr bool all_forward =
      (ForwardRange<maybe_const<Rs>> && ...);
   static_assert(all_forward);
   static_assert(Same<index_sequence_for<Rs...>,
      index_sequence<Is...>>);

   tuple<sentinel_t<maybe_const<Rs>>...> last_;
public:
   __sentinel() = default;
   explicit constexpr __sentinel(Parent& parent);
};
```

4.2. `view::zip`

The name `view::zip` denotes a range view adaptor. Let `rs...` denote a pack expansion of the parameter pack `Rs...`. Then, the expression `view::zip(rs...)` is expression-equivalent to:

1. `zip_view{rs...}` if `(Range<Rs> && ...) && (is_object_v<Rs> && ...)` is satisfied.
2. Otherwise `view::zip(rs...)` is ill-formed.

4.3. `basic_istream_view`

4.3.1. Motivation

`istream_iterator` is an abstraction over a `basic_istream` object, so that it may be used as though it were an input iterator. It is a great way to populate a container from the get-go, or fill a container later in its lifetime. This is great, as copy is a standard algorithm that cleanly communicates that we're copying something from one range into another. There aren't any Hidden Surprises™. This is also good when writing generic code, because the generic library author does not need to care how things are inserted at the end of `v`, only that they are.

Without `istream_iterator`

```
auto v = []{
   auto result = vector<int>{};
   for (auto i = 0; cin >> i;) {
      result.push_back(i);
   }
}();
// ...
for (auto i = 0; cin >> i;) {
   result.push_back(i);
}
```

With `istream_iterator`

```
auto v = vector(istream_iterator<int>{cin},
   istream_iterator<int>{});
// ...
copy(istream_iterator<int>{cin},
   istream_iterator<int>{},
   back_inserter(v));
```

The problem with `istream_iterator` is that we need to provide a bogus `istream_iterator<T>{}` (or `default_sentinel{}`) every time we want to use it; this acts as the sentinel for `istream_iterator`. It is bogus, because the code is equivalent to saying "from the first element of the istream range until the last element of the istream range", but an `istream` range does not have a last element. Instead, we denote the end of an istream range to be when the `istream`'s failbit is set. This is otherwise known as the *end-of-stream* iterator value, but it doesn't denote a 'past-the-last element' in the same way that call to `vector<T>::end` does. Because it is the same for every range, the *end-of-stream* object may as well be dropped, so that we can write code that resembles the code below.

```
auto v = vector(ranges::istream_view<int>{cin});
// ...
copy(ranges::istream_view<int>{cin}, back_inserter(v));
```

This code is cleaner: we are implicitly saying "until our `basic_istream` object fails, insert our input into the back of `v`". There is less to read (and type!), and the code is simpler for it.

4.3.2. Interface

```
template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept StreamExtractable = see-below;

template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept StreamInsertable = see-below;

template<Semiregular Val, class CharT, class Traits = char_traits<CharT>>
requires StreamExtractable<Val, CharT, Traits>
class basic_istream_view : public view_interface<basic_istream_view<Val, CharT, Traits>> {
public:
   basic_istream_view() = default;

   explicit constexpr basic_istream_view(basic_istream<CharT, Traits>& stream);

   constexpr auto begin();
   constexpr default_sentinel end() const noexcept;
private:
   struct __iterator; // exposition-only
   basic_istream<CharT, Traits>* stream_; // exposition-only
   Val object_; // exposition-only
};

template<class T, class CharT, class Traits>
basic_istream_view<T, CharT, Traits> istream_view(basic_istream<CharT, Traits>& in);
```

4.3.2.1. Concept `StreamExtractable`

```
template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept StreamExtractable =
   requires(basic_istream<CharT, Traits>& is, T& t) {
      { is >> t } -> Same<basic_istream<CharT, Traits>>&;
   };
```

1. *Remarks*: `std::addressof(is) == std::addressof(is << t)`.

4.3.2.2. Concept `StreamInsertable`

```
template<class T, class CharT = char, class Traits = char_traits<CharT>>
concept StreamInsertable =
   requires(basic_ostream<CharT, Traits>& os, const T& t) {
      { os << t } -> Same<basic_ostream<CharT, Traits>>&;
   };
```

2. *Remarks*: `std::addressof(os) == std::addressof(os >> t)`.

4.3.2.3. `basic_istream_view` constructor

```
explicit constexpr basic_istream_view(basic_istream<CharT, Traits>& stream);
```

3. *Effects*: Initialises `stream_` to `std::addressof(stream)`.

4.3.2.4. `basic_istream_view` *begin*

```
constexpr auto begin();
```

4. *Effects*: Equivalent to

```
*stream_ >> object_;
return __iterator{*this};
```

4.3.2.5. `basic_istream_view` *end*

```
constexpr default_sentinel end() const noexcept;
```

5. *Returns*: `default_sentinel{}`.

4.3.2.6. `basic_istream_view::__iterator`

```
template<class Val, class CharT, class Traits>
class basic_istream_view<Val, CharT, Traits>::__iterator {
public:
   using iterator_category = input_iterator_tag;
   using difference_type = ptrdiff_t;
   using value_type = Val;

   __iterator() = default;

   explicit constexpr __iterator(basic_istream_view<Val>& parent) noexcept;

   __iterator& operator++();
   void operator++(int);

   Val& operator*() const;

   friend bool operator==(__iterator x, default_sentinel);
   friend bool operator==(default_sentinel y, __iterator x);
   friend bool operator!=(__iterator x, default_sentinel y);
   friend bool operator!=(default_sentinel y, __iterator x);
private:
   basic_istream_view<Val, CharT, Traits>* parent_ = nullptr; // exposition-only
};
```

4.3.2.7. `basic_istream_view::__iterator` *constructor*

```
explicit constexpr __iterator(basic_istream_view<Val>& parent) noexcept;
```

6. *Effects*: Initialises `parent_` with `std::addressof(parent_)`.

4.3.2.8. `basic_istream_view::__iterator` *next*

```
__iterator& operator++();
void operator++(int);
```

7. *Effects*: Equivalent to

```
*parent_->stream_ >> parent_->object_;
```

§ 4.3.2.9. `basic_istream_view::__iterator` *value*

```
Val& operator*() const;
```

8. *Effects*: Equivalent to `return parent_->value_;`

§ 4.3.2.10. `basic_istream_view::__iterator` *comparison functions*

```
friend bool operator==(__iterator x, default_sentinel);
```

9. *Effects*: Equivalent to `return !*x.parent_->stream_;`.

```
friend bool operator==(default_sentinel y, __iterator x);
```

10. *Returns*: `x == y`.

```
friend bool operator!=(__iterator x, default_sentinel y);
```

11. *Returns*: `!(x == y)`.

```
friend bool operator!=(default_sentinel y, __iterator x);
```

12. *Returns*: `!(x == y)`.

§ 4.3.2.11. `basic_istream_view` *factory*

```
template<class T, class CharT, class Traits>
basic_istream_view<T, CharT, Traits> istream_view(basic_istream<CharT, Traits>& s);
```

13. *Effects*: Equivalent to `return basic_istream_view<T, CharT, Traits>{s};`.

§ 4.4. Introducing associated types for ranges

Note: This section was formerly titled *Reviewing `iter_value_t`, etc., and introducing `range_value_t`, etc.* in P1035R1, but the current title is more accurate.

[P1037] introduced `iter_difference_t`, `iter_value_t`, `iter_reference_t`, and `iter_rvalue_reference_t`, which dispatch to templates that correclty identify the associated types for iterators (formerly known as iterator traits). The current mechanism supports iterators, but not ranges. For example, in order to extract an range's value type, we are required to do one of three things:

1. Use the traditional `typename T::value_type`. As we have had template aliases since C++11, this has probably been abandoned in many projects (not cited).
2. Define our own associated type à la

```
template<class T>
requires requires { typename T::value_type; }
using value_type = typename T::value_type;
```

This was apparently possible in the Ranges TS (albeit in much more detail) [iter.assoc.types.value_type], but P1037 has redefined `value_type_t` as `iter_value_t`, which focuses on iterators, rather than anything defining `value_type` alias. We *could* do this in C++20, but we then risk `R::value_type` being different to `R::iterator::value_type` (consider `vector<int const>::value_type` and `vector<int const>::iterator::value_type`, which are `int const` and `int`, respectively)[value_type].

3. Use `iter_value_t<iterator_t<R>>`, which what we probably want, and is also a mouthful.

When discussing extending `iter_value_t` so that it can also operate on ranges, Casey Carter had this to say:

> What should `iter_value_t<T>` be when `T` is an iterator with value type `U` and a range with value type `V`?
>
> Alternately: What is `iter_value_t<array<const int>>`? `array<const int>::value_type` is `const int`, but `iter_value_t<iterator_t<array<const int>>` is `int`.
>
> Specific examples aside, early attempts to make the associated type aliases "polymorphic" kept running into ambiguities that we had to disambiguate or simply declare broken.
>
> I'd rather see `range_value_t` et al proposed.

P1035 introduces shorthands for `iter_foo_t<iterator_t<R>>` as `range_foo_t<R>`, which in turn have been taken from range-v3.

```
template<Range R>
using range_difference_t = iter_difference_t<iterator_t<R>>;

template<Range R>
using range_size_t = make_unsigned_t<range_difference_t<R>>;

template<InputRange R>
using range_value_t = iter_value_t<iterator_t<R>>;

template<InputRange R>
using range_reference_t = iter_reference_t<iterator_t<R>>;

template<InputRange R>
using range_rvalue_reference_t = iter_rvalue_reference_t<iterator_t<R>>;

template<Range R>
requires Same<iterator_t<R>, sentinel_t<R>>
using range_common_iterator_t = common_iterator<iterator_t<R>, sentinel_t<R>>;
```

These have an enormous amount of usage in range-v3, and will ensure consistency between generic code that uses ranges and generic code that uses iterators (which are essential for underpinning all range abstractions in C++).

4.5. `take_while_view`

4.5.1. Motivation

P0789 introduces `take_view`, which rangifies the iterator pair `{ranges::begin(r), ranges::next(ranges::begin(r), n, ranges::end(r))}`. As an example:

```
auto v = vector{0, 1, 2, 3, 4, 5};
cout << distance(v | view::take(3)) << '\n'; // prints 3
copy(v | view::take(3), ostream_iterator<int>(cout, " ")); // prints 0 1 2
copy(v | view::take(distance(v)), ostream_iterator<int>(cout, " ")); // prints 0 1 2 3 4 5
```

`take_while_view` will provide slightly different functionality, akin to having a sentinel that checks if a certain predicate is satisfied.

Current

```
namespace ranges = std::experimental::ranges;

template <ranges::Integral I>
constexpr bool is_odd(I const i) noexcept
{
   return i % 2;
}

struct is_odd_sentinel {
   template <ranges::Iterator I>
   constexpr friend bool operator==(I const& a,
      is_odd_sentinel) noexcept
   {
      return is_odd(*a);
   }

   template <ranges::Iterator I>
   constexpr friend bool operator==(is_odd_sentinel const a,
      I const& b) noexcept
   {
      return b == a;
   }

   template <ranges::Iterator I>
   constexpr friend bool operator!=(I const& a,
      is_odd_sentinel const b) noexcept
   {
      return not (a == b);
   }

   template <ranges::Iterator I>
   constexpr friend bool operator!=(is_odd_sentinel const a,
      I const& b) noexcept
   {
      return not (b == a);
   }
};

int main()
{
   auto v = vector{0, 6, 8, 9, 10, 16};
   ranges::copy(
     ranges::begin(v),
     is_odd_sentinel{},
     ranges::ostream_iterator<int>(cout, "\n"));
}
```

Wandbox demo

Proposed

```
namespace ranges = std::experimental::ra

template <ranges::Integral I>
constexpr bool is_odd(I const i) noexcep
{
   return i % 2;
}

template <ranges::Integral I>
constexpr bool is_even(I const i) noexce
{
   return not is_odd(i);
}

int main()
{
   namespace view = ranges::view;
   auto v = vector{0, 6, 8, 9, 10, 16};
   ranges::copy(
      v | view::take_while(is_even<int>)
      ranges::ostream_iterator<int>(cout
}
```

Wandbox demo

The former requires that a user define their own sentinel type: something that while not expert-friendly, is yet to be established as a widespread idiom in C++, and providing a range adaptor for this purpose will help programmers determine when a sentinel is _not_ necessary.

§ 4.5.2. Notes

- There is a slight programmer overhead in the naming: the author felt that both `is_odd_sentinel` and `is_even_sentinel` were applicable names: ultimately, the name `is_odd_sentinel` was chosen because it describes the delimiting condition. An equally valid reason could probably be made for `is_even_sentinel`.

- A sentinel that takes a lambda may be of interest to LEWG. If there is interest in this, a proposal could be made in the C++23 timeframe.

4.5.3. Interface and specification

```
template <View R, class Pred>
requires
   InputRange<R> &&
   is_object_v<Pred> &&
   IndirectUnaryPredicate<const Pred, iterator_t<R>
class take_while_view : public view_interface<take_while_view<R, Pred>> {
   template<bool> class __sentinel; // exposition-only
public:
   take_while_view() = default;
   constexpr take_while_view(R base, Pred pred);

   template <ViewableRange O>
   requires constructible-from-range<R, O>
   constexpr take_while_view(O&& o, Pred pred);

   constexpr R base() const;
   constexpr const Pred& pred() const;

   constexpr auto begin() requires (!simple-view<R>);
   constexpr auto begin() const requires Range<const R>;

   constexpr auto end() requires (!simple-view<R>);
   constexpr auto end() const requires Range<const R>;
private:
   R base_; // exposition-only
   semiregular<Pred> pred_; // exposition-only
};

template<class R, class Pred>
take_while_view(R&&, Pred) -> take_while_view<all_view<R>, Pred>;
```

4.5.3.1. `take_while_view` constructors

```
constexpr take_while_view(R base, Pred pred);
```

1. *Effects*: Initialises `base_` with `base` and initialises `pred_` with `pred`.

```
template <ViewableRange O>
requires constructible-from-range<R, O>
constexpr take_while_view(O&& o, Pred pred);
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))` and initialises `pred_` with `pred`.

4.5.3.2. `take_while_view` conversion

```
constexpr R base() const;
```

1. *Returns*: `base_`.

```
constexpr const Pred& pred() const;
```

2. *Returns*: `pred_`.

§ *4.5.3.3. `take_while_view` range begin*

```
constexpr auto begin() requires (!simple-view<R>);
constexpr auto begin() const requries Range<const R>
```

1. *Effects*: Equivalent to `return ranges::begin(base_);`.

§ *4.5.3.4. `take_while_view` range end*

```
constexpr auto end() requires (!simple-view<R>);
constexpr auto end() const requires Range<const R>;
```

1. *Effects*: Equivalent to `return __sentinel<is_const_v<decltype(*this)>>(&pred());`.

§ **4.5.4. `take_while_view::__sentinel`**

```
template<class R, class Pred>
template<bool Const>
class take_while_view<R, Pred>::__sentinel {
   using Parent = conditional_t<Const, const take_while_view, take_while_view>;
   using Base = conditional_t<Const, const R, R>;
   sentinel_t<Base> end_{}; // exposition-only
   const Pred* pred_{}; // pred
public:
   __sentinel() = default;
   constexpr explicit __sentinel(sentinel_t<Base> end, const Pred* pred);
   constexpr __sentinel(__sentinel<!Const> s)
     requires Const && ConvertibleTo<sentinel_t<R>, sentinel_t<Base>>

   constexpr sentinel_t<Base> base() const { return end_; }

   friend constexpr bool operator==(const __sentinel& x, const iterator_t<Base>& y);
   friend constexpr bool operator==(const iterator_t<Base>& x, const __sentinel& y);
   friend constexpr bool operator!=(const __sentinel& x, const iterator_t<Base>& y);
   friend constexpr bool operator!=(const iterator_t<Base>& x, const __sentinel& y);
};
```

§ **4.5.5. `take_while_view::__sentinel` constructor**

```
constexpr explicit __sentinel(sentinel_t<Base> end, const Pred* pred);
```

1. *Effects*: Initialises `end_` with `end`, and `pred_` with `pred`.

```
constexpr __sentinel(__sentinel<!Const> s)
   requires Const && ConvertibleTo<sentinel_t<R>, sentinel_t<Base>>;
```

2. *Effects* Initialises `end_` with `s.end_` and `pred_` with `s.pred_`.

4.5.6. `take_while_view::__sentinel` conversion

```
constexpr sentinel_t<Base> base() const;
```

3. *Effects*: Equivalent to `return end_;`

4.5.7. `take_while_view::__sentinel` comparisons

```
friend constexpr bool operator==(const __sentinel& x, const iterator_t<Base>& y)
```

4. *Effects*: Equivalent to `return x.end_ != y && !(*x.pred_)(*y);`.

```
friend constexpr bool operator==(const iterator_t<Base>& x, const __sentinel& y);
```

5. *Effects*: Equivalent to `return y == x;`.

```
friend constexpr bool operator!=(const __sentinel& x, const iterator_t<Base>& y);
```

6. *Effects*: Equivalent to `!(x == y);`.

```
friend constexpr bool operator!=(const iterator_t<Base>& x, const __sentinel& y);
```

7. *Effects*: Equivalent to `!(y == x);`.

4.6. `view::take_while`

The name `view::take_while` denotes a range adaptor object. Let `E` and `F` be expressions such that type `T` is `decltype((E))`. Then, the expression `view::take_while(E, F)` is expression-equivalent to:

1. `take_while_view{E, F}` if `T` models `InputRange` and if `F` is an object, and models `IndirectUnaryPredicate`.
2. Otherwise `ranges::view::take_while(E, F)` is ill-formed.

4.7. `drop_view`

4.7.1. Motivation

`drop_view` is the complement to `take_view`: instead of providing the user with the first *n* elements, it provides the user with all *but* the first *n* elements.

Current (C++17)	
`auto begin = next(begin(employees), 5, end(employees));` `auto result = find(begin, end(employees), "Lovelace", &employees::surname);`	`namespace view = ranges::` `auto result = find(employ`

4.7.2. Interface

```
template<View R>
class drop_view : public view_interface<drop_view<R>> {
   using D = iter_difference_t<iterator_t<R>>; // exposition-only
public:
   drop_view();
   constexpr drop_view(R base, D count) [[expects: 0 < count]];

   template<ViewableRange O>
   requires constructible-from-range<R, O>
   constexpr drop_view(O&& o, D count) [[expects: 0 < count]];

   constexpr R base() const;

   constexpr auto begin() requires (!(simple-view<R> && RandomAccessRange<R>));
   constexpr auto begin() const requires Range<const R> && RandomAccessRange<const R>;
   constexpr auto end() requires (!simple-view<R>);
   constexpr auto end() const requires Range<const R>;

   constexpr auto size() requires (!simple-view<R>) && SizedRange<R>;
   constexpr auto size() const requires SizedRange<const R>;
private:
   R base_; // exposition-only
   D count_; // exposition-only
};

template<Range R>
drop_view(R&&, iter_difference_t<iterator_t<R>>) -> drop_view<all_view<R>>;
```

4.7.2.1. `drop_view` constructor

```
constexpr drop_view(R base, D count) [[expects: 0 < count]];
```

1. *Effects*: Initialises `base_` with `base` and `count_` with `count`.

```
template<ViewableRange O>
requires constructible-from-range<R, O>
constexpr drop_view(O&& o, D count) [[expects: 0 < count]];
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))` and `count_` with `count`.

4.7.2.2. `drop_view` conversion

```
constexpr R base() const;
```

3. *Effects*: Equivalent to `return base_`.

4.7.2.3. `drop_view` range begin

```
constexpr auto begin() requires (!(simple-view<R> && RandomAccessRange<R>));
constexpr auto begin() const requires Range<const R> && RandomAccessRange<const R>;
```

4. *Effects*: Equivalent to `return ranges::next(ranges::begin(base_), count_, ranges::end(base_));`.

5. *Remarks*: In order to provide the amortized constant time complexity required by the Range concept, the first overload caches the result within the `drop_view` for use on subsequent calls.

4.7.2.4. `drop_view` *range end*

```
constexpr auto end() requires (!simple-view<R>);
constexpr auto end() const requires Range<const R>;
```

6. *Effects*: Equivalent to `return ranges::end(base_);`.

4.7.2.5. `drop_view` *size*

```
constexpr auto size() requires (!simple-view<R>) && SizedRange<R>;
constexpr auto size() const requires SizedRange<const R>;
```

7. Equivalent to:

```
auto const size = ranges::size(base_);
auto const count = static_cast<decltype(size)>(count_);
return size < count ? 0 : size - count;
```

4.8. `view::drop`

The name `view::drop` denotes a range adaptor object. Let `E` and `F` be expressions such that type `T` is `decltype((E))`. Then, the expression `view::drop(E, F)` is expression-equivalent to:

1. `drop_view{E, F}` if `T` models `InputRange` and `F` is implicitly convertible to `iter_difference_t<iterator_t<T>>`.
2. Otherwise `view::drop(E, F)` is ill-formed.

4.9. `drop_while_view`

4.9.1. Motivation

The motivation for `drop_while_view` is the union of `drop_view` and `take_while_view`.

Current (C++17)	P

`auto begin = find_if_not(employees,`
`  [](auto const holidays){ return holidays < 20; },`
`  &employee::holidays);`
 	
`transform(begin, end(employees),`
`  ostream_iterator<string>(cout, "\n"),`
`  [](auto const& e) { return e.given_name() + e.surname(); });`
	`auto excess_holidays = employees`
	`  | view::drop_whi`
	`transform(excess_holidays, ostream_it`
	`  [](auto const& e) { return e.given`

4.9.2. Interface

```
template<View R, class Pred>
requires
  InputRange<R> &&
  is_object_v<Pred> &&
  IndirectUnaryPredicate<const Pred, iterator_t<R>>
class drop_while_view : public view_interface<drop_while_view<R, Pred>> {
public:
   drop_while_view() = default;

   constexpr drop_while_view(R base, Pred pred);

   template<ViewableRange O>
   requires constructible-from-range<R, O>
   constexpr drop_while_view(O&& o, Pred pred);

   constexpr R base() const;
   constexpr Pred pred() const;

   constexpr auto begin();
   constexpr auto end();
private:
   R base_; // exposition-only
   semiregular<Pred> pred_; // exposition-only
};

template<class R, class Pred>
drop_while_view(R&&, Pred) -> drop_while_view<all_view<R>, Pred>;
```

4.9.2.1. `drop_while_view` constructors

```
constexpr drop_while_view(R base, Pred pred);
```

1. *Effects*: Initialises `base_` with `base` and initialises `pred_` with `pred`.

```
template<ViewableRange O>
requires constructible-from-range<R, O>
constexpr drop_while_view(O&& o, Pred pred)
```

2. *Effects*: Initialises `base_` with `view::all(std::forward<O>(o))`, and intialises `pred_` with `pred`.

4.9.2.2. `drop_while_view` conversion

```
constexpr R base() const;
```

3. *Returns*: `base_`.

```
constexpr Pred pred() const;
```

4. *Returns*: `pred_`.

§ 4.9.2.3. `drop_while_view` *begin*

```
constexpr auto begin();
```

5. *Effects*: Equivalent to `return ranges::find_if_not(base_, std::ref(pred_));`.
6. *Remarks*: In order to provide the amortized constant time complexity required by the `Range` concept, the first overload caches the result within the `drop_while_view` for use on subsequent calls.

§ 4.9.2.4. `drop_while_view` *end*

7. *Effects*: Equivalent to `return ranges::end(base_);`.

§ 4.10. `view::drop_while`

The name `view::drop_while` denotes a range adaptor object. Let `X` and `Y` be expressions such that type `T` is `decltype((X))` and `F` is `decltype((Y))`. Then, the expression `view::drop_while(X, Y)` is expression-equivalent to:

1. `drop_while_view{X, Y}` if `T` models `InputRange`, and `Y` is an object, and `F`| and models `IndirectUnaryPredicate`.
2. Otherwise `view::drop_while(E, F)` is ill-formed.

§ 4.11. `keys` and `values`

§ 4.11.1. Motivation

It is frequent to want to iterate over the keys or the values of an associative container. There is currently no easy way to do that. It can be approximated using `ranges::view::transform`, given how frequently such operation is needed we think their addition would make manipulation of associative containers easier by more clearly expressing the intent.

These views have been part of ranges-v3 and we also have implemented them in cmcstl2. A lot of languages and frameworks (notably JS, Python, Qt, Java, [Boost.Range]) offer methods to extract the keys or values of an associative container, often through `.keys()` and `.values()` operations, respectively.

§ 4.11.2. Implementation

Given an exposition-only tuple extractor (`__get_fn`), `keys` and `values` need only be range adaptor objects implemented in terms of `view::transform`, rather than range adaptors.

```
namespace view {
   template<int X>
   requires (X == 0) || (X == 1)
   struct __get_fn {
      template<class P>
      requires pair-like<P>
      constexpr decltype(auto) operator()(P&& p) const
      {
         using T = decltype(std::get<X>(std::forward<P>(p)));
         return std::get<X>(std::forward<P>(p));
      }
   };

   inline constexpr auto keys = view::transform(__get_fn<0>{});
   inline constexpr auto values = view::transform(__get_fn<1>{});
} // namespace view
```

§ References

§ Informative References

[Boost.Range]
Boost.Range. URL: https://www.boost.org/doc/libs/1_68_0/libs/range/doc/html/range/reference/adaptors/reference/map_values.html#range.reference.adaptors.reference.map_values.map_values_example

[CMCSTL2]
Casey Carter & others. cmcstl2/zip_view. URL: https://github.com/caseycarter/cmcstl2/tree/zip_view

[CONVOLUTION]
Wikipedia. Convolution. URL: https://en.wikipedia.org/wiki/Convolution_(computer_science)

[EWG43]
Gabriel Dos Reis. [tiny] simultaneous iteration with new-style for syntax. URL: https://cplusplus.github.io/EWG/ewg-active.html#43

[ITER.ASSOC.TYPES.VALUE_TYPE]
Eric Niebler; Casey Carter. ISO/IEC TS 21425:2017 C++ Extensions for Ranges. URL: https://timsong-cpp.github.io/cppwp/ranges-ts/iterator.assoc.types.value_type

[P0026]
Matthew McAtamney-Greenwood. multi-range-based for loops. URL: https://wg21.link/p0026

[P0789]
Eric Niebler. Range Adaptors and Utilities. URL: https://wg21.link/p0789

[P0836]
Gordon Brown; et al. P0836 Introduce Parallelism to the Ranges TS. URL: https://wg21.link/p0836

[P1037]
Eric Niebler; Casey Carter. Deep Integration of the Ranges TS. URL: https://wg21.link/p1037

[RANGE-V3]
Eric Niebler & others. range-v3. URL: https://github.com/ericniebler/range-v3

[VALUE_TYPE]
static_assert(not is_same_v<vector<int const>::value_type, vector<int const>::iterator::value_type>). URL: https://godbolt.org/z/xkcREX